West Kenya Union Conference

Deployment Report Mini-Enterprise Infrastructure Project

Student Name: Jedidah Nyandoro
Station: WKUC HQ Station 1
Project: Mini-Enterprise Infrastructure Project
Report Date:
Supervisor: John Marande

1. **Summary**

The Mini-Enterprise Infrastructure Project is a deployment covering virtualization and Networking setup, Linux Systems Administration and hardening, Automation, Cloud Integration and Backup Strategy, testing, monitoring and documentation. Focusing on deploying a secure, automated, and backed-up web application environment.

2. **Project Objectives**

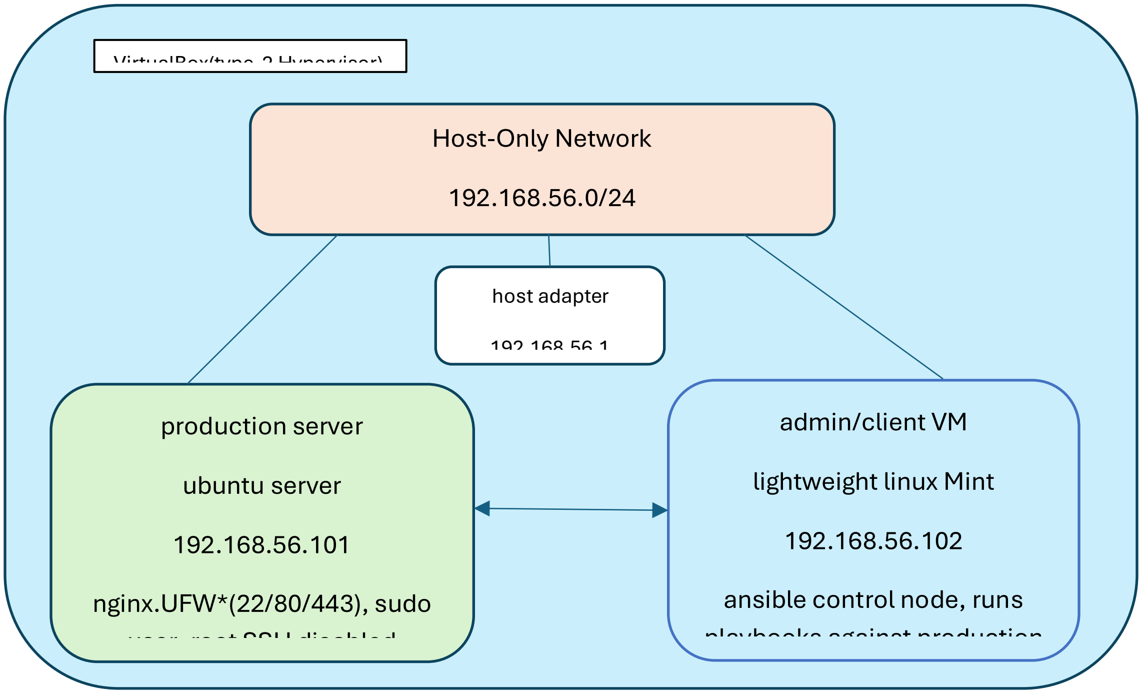
- Establish an isolated virtual lab environment
- Deploy and harden a Linux production server
- Automate deployment using Ansible
- Implement cloud-based backup and disaster recovery
- Validate the environment through testing and monitoring

3. **Network Architecture**

The lab environment is built on an isolated Host-Only network. A host-only network is used because it provides a completely isolated, secure sandbox that guarantees static IP stability for reliable automation without interfering with your physical network.

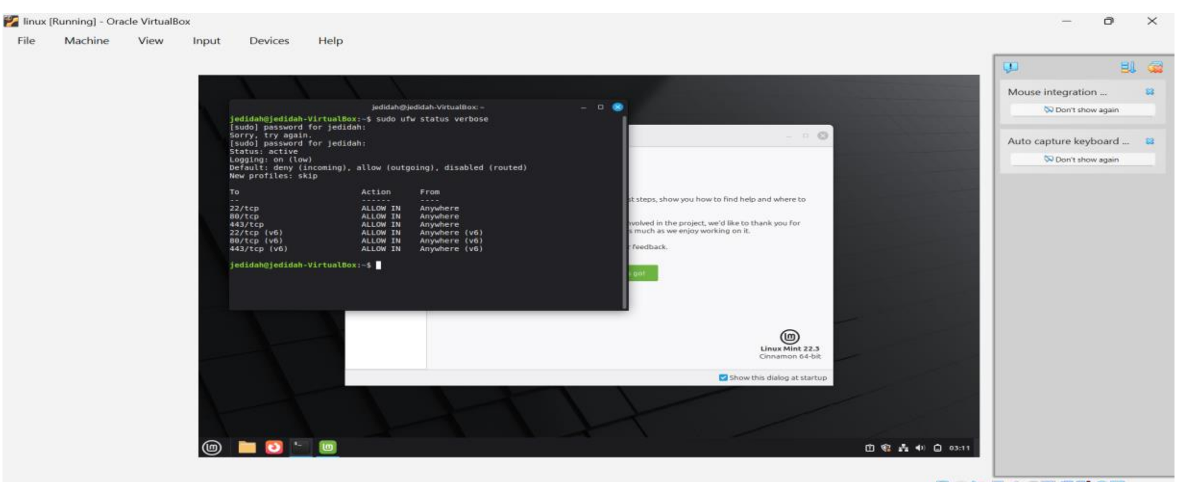
Field	**Value**
Hypervisor	VirtualBox (Type-2)
Host Machine	HP EliteBook 840 G6 Windows 11, 8GB RAM
Network Type	Host-Only Adapter
Subnet	192.168.56.0/24
Host Adapter IP	192.168.56.1
Production Server (Ubuntu)	192.168.56.101
Admin/Client VM (Linux mint)	192.168.56.102

3. **Network Diagram**

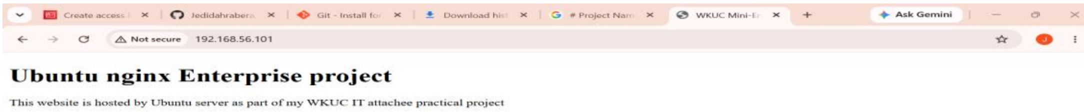


4. **Systems Administration & Hardening**

- OS packages updated and upgraded (apt update && apt upgrade)
- Dedicated sudo user created, root SSH login disabled.
- UFW firewall restricted to ports 22 (SSH), 80 (HTTP), 443 (HTTPS)
- Nginx installed and verified serving a custom landing page



Ufw status output

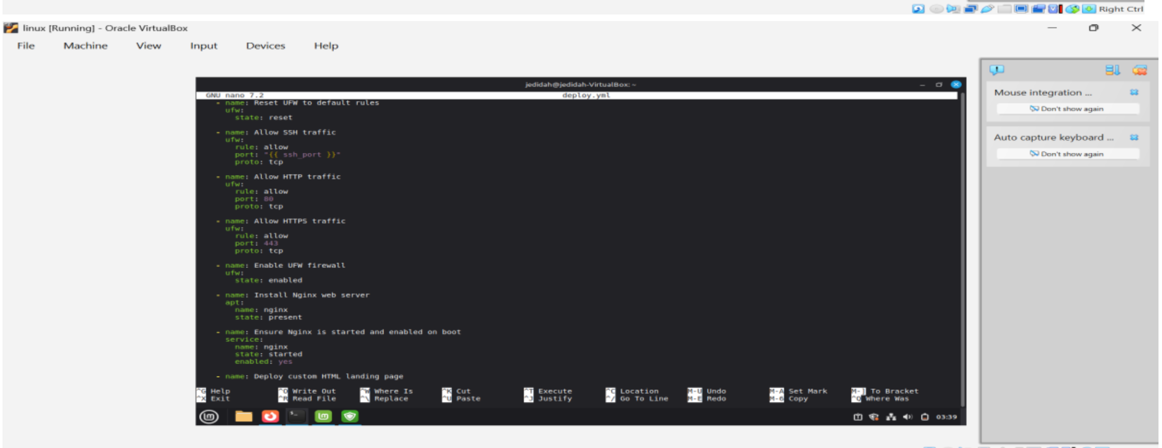


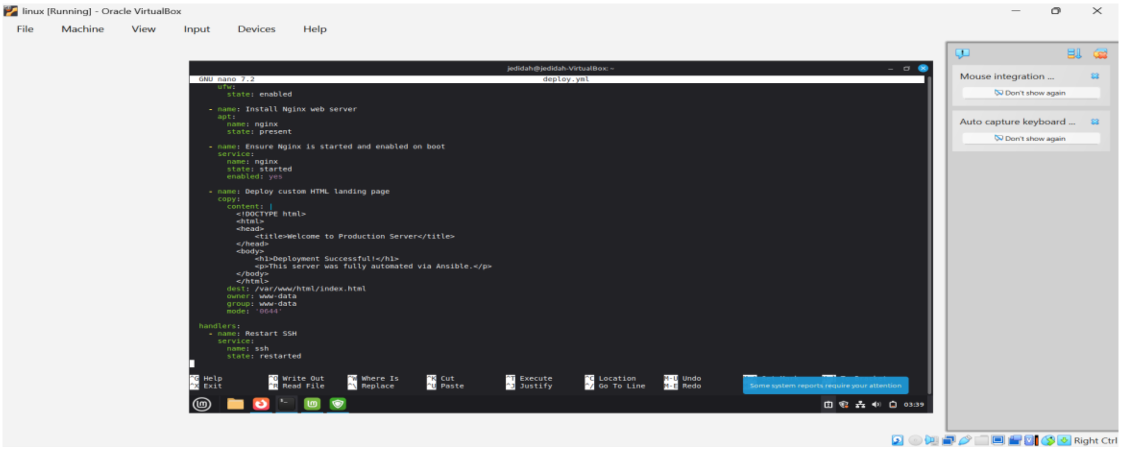
Website loading in browser

5. **Automation** (Infrastructure as Code)

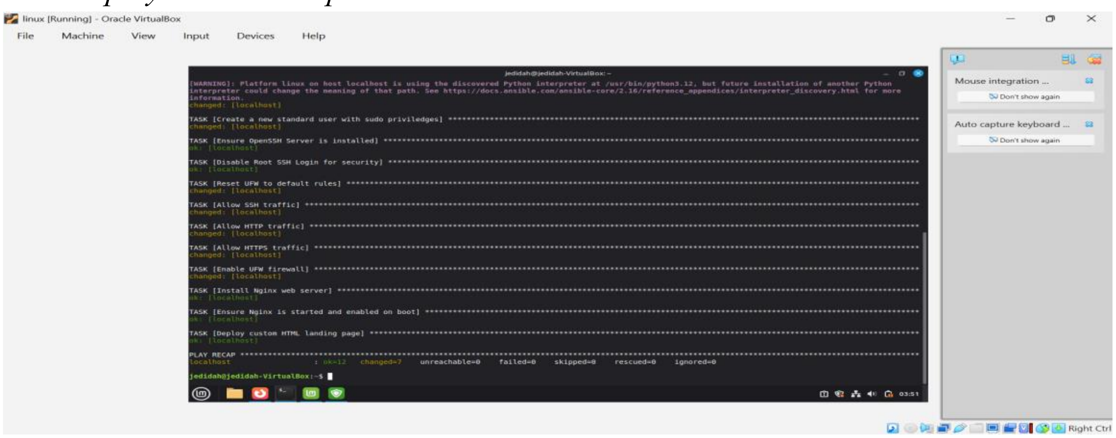
Manual configuration from the hardening phase was replaced with an Ansible playbook, allowing the entire production environment to be reproduced automatically on any fresh VM.

<u>*Ansible playbook*</u>





Ansible playbook-run output



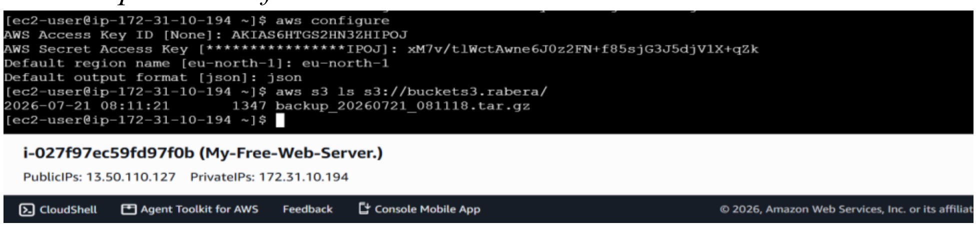
6. **Cloud Integration & Backup Strategy**

To protect against data loss on the production server, a cloud-based backup strategy was implemented using AWS S3. A storage bucket named [buckets3.rabera] was created under a free-tier account, with an IAM user configured with least privilege access restricted to s3: PutObject and s3: GetObject permissions only.

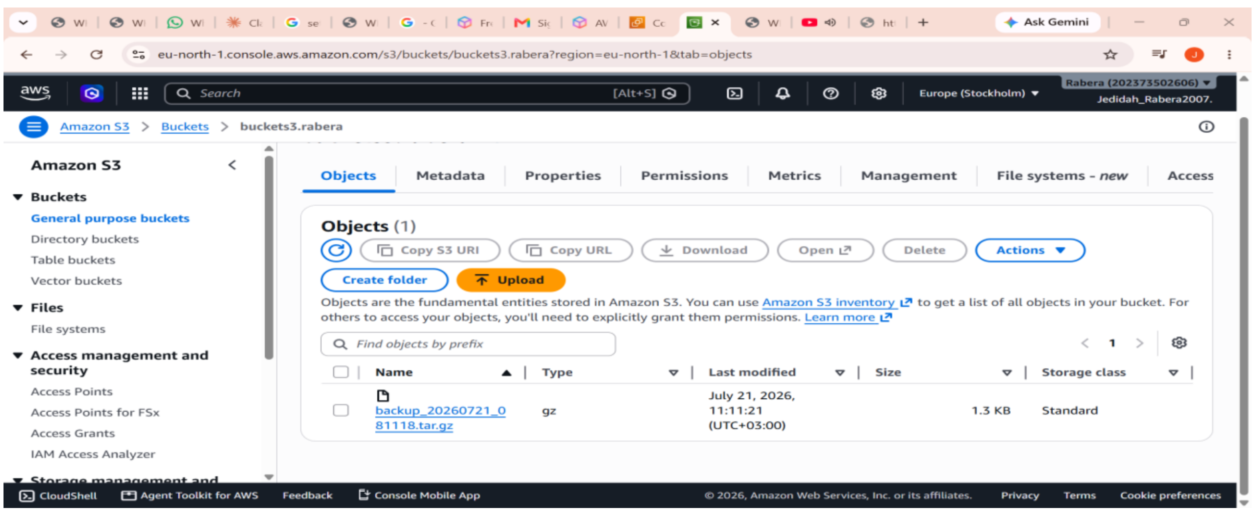
A bash script (backup.sh) was written on the production server to compress the website files *(/var/www/html)* and Nginx logs *(/var/log/nginx)* into a single dated.tar.gz archive. The AWS CLI was configured on the server using the IAM access keys, and the script uploads each archive directly to the S3 bucket via aws s3 cp. To ensure backups run without manual intervention, a cron job was scheduled to execute the script nightly at midnight (0 0 * * *). This establishes an automated, recurring backup cycle independent of any single point of failure on the local VM.

Backup Script Bash

Cloud Upload Proof



S3 bucket console

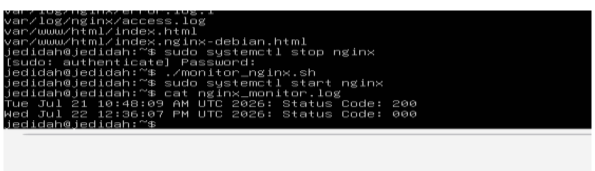


7. **Disaster Recovery**

To validate that the cloud backup strategy provides genuine data resilience, a controlled disaster recovery test was performed on the production server. The website's primary HTML file (/var/www/html/index.html) was deliberately deleted to simulate data loss, after which the most recent backup archive was retrieved from the AWS S3 bucket (buckets3.rabera) and used to restore the site to full working order.

The test confirmed that the backup pipeline established in Section 6 is not merely configured but functionally reliable: the deleted file was successfully recovered from backup_20260721_081118.tar.gz, and the website returned to normal operation on the first restore attempt. This exercise validates the disaster recovery capability of the environment and confirms that a real data-loss incident could be resolved using the existing backup process without additional troubleshooting.

8. **Monitoring Bash**



9. **Challenges & Resolutions**

Challenge: Minor VM/network configuration adjustments while assigning static IPs and getting both VMs communicating on the Host-Only network.
Resolution: Adjusted Host-Only network settings and static IP assignments via Netplan until both VMs can ping each other successfully.
Challenge: Hardware issue- HP laptop experienced intermittent hangs.
Challenge: Missing credentials- First backup attempt failed with Unable to locate credentials;
Resolution: Running aws configure.
Challenge: Command syntax- A missing space in the aws s3 cp command caused a "paths required" error.
Resolution: Corrected the spacing.
Challenge: Disaster test (over-deletion): Deleted the entire /var/www/html/ directory instead of just index.html; recreated the directory before restoring a more thorough test than planned.
Resolution: AWS access keys were visible in a few screenshots during documentation; the exposed key was deactivated, deleted, and replaced immediately.

Challenge: New credentials returned SignatureDoesNotMatch due to a mistyped secret key. resolved by re-entering it via copy-paste instead of manual typing.
Resolution: Manually forced hardware clock time synchronization alignments utilizing: `sudo date -s "15:02:00"` and `sudo hwclock --hctosys` to instantly normalize the server timeline.
Challenge: Cloud Access Denied Violations (`403 Forbidden`).
Resolution: Re-entered clean programmatic credentials securely associated with IAM user `Ida@Nyandoro`, validating correct `s3:GetObject` privileges.

10. **Conclusion**

- Successfully completed all five days of the Mini-Enterprise Infrastructure Project, meeting every stated objective.
- Provisioned and networked two VMs on an isolated Host-Only network (192.168.56.0/24) with static IP addressing.
- Hardened the Ubuntu production server: sudo-only access, root SSH disabled, UFW restricted to ports 22/80/443, Nginx deployed and verified.
- Automated the entire hardening and deployment process using an Ansible playbook, proving the environment could be reproduced reliably on a fresh VM.
- Implemented a cloud backup strategy using AWS S3, with a scripted, cron-scheduled nightly upload of website files and logs.
- Validated the backup strategy through a real disaster recovery test — deliberately deleting production data and successfully restoring it from the S3 backup.
- Encountered and resolved several real-world issues, including credential misconfiguration, command syntax errors, and a brief AWS key exposure, without compromising system integrity.
- Gained hands-on experience across the full infrastructure lifecycle: virtualization, networking, systems administration, automation, cloud integration, and disaster recovery.